

Panduan Mengisi Tabel Evaluasi Offline (Tabel 8 & Tabel 9)

Dokumen ini menjelaskan **cara mengisi tabel kosong** pada BAB IV pengujian, serta **rumus perhitungan** yang dipakai sistem (selaras `python/cf/evaluator.py` dan `RecommenderService`).

1. Apa yang perlu diisi?

Tabel	Isi	Sumber angka
Tabel 8	Precision, Recall, F1, Hit Rate, Catalog Coverage untuk K=5 dan K=10	Output python <code>-m cf.evaluator</code> atau tabel <code>evaluation_logs</code>
Tabel 9	Metadata: tanggal, commit, jumlah data, users dievaluasi, durasi	SQL + output evaluator + <code>git</code>

Metode yang ditulis di Tabel 8: **IBCF Cosine** (bukan Most Popular / NDCG).

2. Prasyarat sebelum menjalankan

- MySQL berisi transaksi valid: `status_pesanan = 'Selesai'` **dan** `status_pembayaran = 'Dibayar'`, produk `status = 'aktif'`.
- Migrasi sudah jalan (`evaluation_logs` ada): `php artisan migrate`
- Dependency Python terpasang:

```
cd d:\PROJECT\rekomendasi_produk_laravel\python pip install -r requirements.txt
```

- Env DB sama dengan Laravel (`.env`):

```
$env:DB_HOST="127.0.0.1" $env:DB_DATABASE="db_sinar_manis" $env:DB_USERNAME="root"
$env:DB_PASSWORD="" $env:CF_MIN_CO_OCCURRENCE="2"
```

Cek data cukup:

```
SELECT COUNT(*) AS trx_valid FROM transactions WHERE status_pesanan = 'Selesai' AND
status_pembayaran = 'Dibayar';
```

Jika `trx_valid = 0`, evaluator tidak menghasilkan angka — isi dulu transaksi selesai/dibayar.

3. Cara menjalankan evaluator

Dari folder `python`:

```
cd d:\PROJECT\rekomendasi_produk_laravel\python python -m cf.evaluator
```

Alternatif (hitung ulang CF + evaluasi):

```
python batch_runner.py
```

Contoh keluaran terminal

```
[EVAL] Precision@5: 0.0046 [EVAL] Recall@5: 0.0228 [EVAL] F1@5: 0.0076 [EVAL] HitRate@5: 0.0228  
[EVAL] CatalogCov@5:94.97% [EVAL] Users: 744 [EVAL] Precision@10: 0.0051 [EVAL] Recall@10: 0.0511  
[EVAL] F1@10: 0.0093 [EVAL] HitRate@10: 0.0511 [EVAL] CatalogCov@10:97.49% [EVAL] Users: 744
```

Atau baca dari DB:

```
SELECT evaluated_at, k_value, users_evaluated, precision_at_k, recall_at_k, f1_at_k, hit_rate_at_k,  
catalog_coverage_at_k, duration_seconds, notes FROM evaluation_logs ORDER BY evaluated_at DESC  
LIMIT 4;
```

4. Cara mengisi Tabel 8

Template:

Metode	K	Precision	Recall	F1	Hit Rate	Catalog Coverage	Users
IBCF Cosine	5	...	...	...	...	...	...
IBCF Cosine	10	...	...	...	...	...	...

Pemetaan output → sel tabel

Kolom Tabel 8	Ambil dari	Contoh isi
Metode	tetap	IBCF Cosine
K	@5 / @10	5 atau 10
Precision	Precision@K	0,0046 atau 0,46%
Recall	Recall@K	0,0228 atau 2,28%
F1	F1@K	0,0076 atau 0,76%
Hit Rate	HitRate@K	0,0228 atau 2,28%
Catalog Coverage	CatalogCov@K	94,97%
Users	Users	744 (sama untuk K=5 dan K=10)

Konsistensi format: pilih satu gaya di seluruh tabel.

- Desimal: 0,0046 (sesuai kolom float di DB)
- Persen: 0,46% (= desimal × 100)

Catalog Coverage di output sudah dalam **persen** (94.97%).

Contoh terisi (ilustrasi — ganti dengan angka run kamu)

Metode	K	Precision	Recall	F1	Hit Rate	Catalog Coverage	Users
IBCF Cosine	5	0,46%	2,28%	0,76%	2,28%	94,97%	744
IBCF Cosine	10	0,51%	5,11%	0,93%	5,11%	97,49%	744

5. Cara mengisi Tabel 9

Template:

Metadata eksekusi	Nilai
Tanggal/waktu evaluasi	...
Commit / versi aplikasi	...
Jumlah transaksi valid (Selesai+Dibayar)	...
Jumlah pelanggan pada interaksi	...
Jumlah produk pada interaksi	...
Users dievaluasi (holdout)	...
min_co_occurrence	2
Durasi (detik)	...

Sumber tiap baris

Field	Cara dapat
Tanggal/waktu	Waktu kamu menjalankan perintah, atau <code>evaluated_at</code> di <code>evaluation_logs</code>
Commit	<code>git rev-parse --short HEAD</code>
Transaksi valid	SQL di bagian 2
Pelanggan & produk	SQL di bawah, atau jalankan <code>load_transaction_data</code>

Users dievaluasi	baris Users dari output evaluator
min_co_occurrence	2 (atau nilai CF_MIN_CO_OCCURRENCE)
Durasi	duration_seconds di evaluation_logs

SQL ringkas untuk metadata:

```
SELECT COUNT(*) AS trx_valid FROM transactions WHERE status_pesanan = 'Selesai' AND
status_pembayaran = 'Dibayar'; SELECT COUNT(DISTINCT t.id_user) AS pelanggan, COUNT(DISTINCT
ti.id_product) AS produk FROM transactions t JOIN transaction_items ti ON ti.id_transaction =
t.id_transaction JOIN products p ON p.id_product = ti.id_product WHERE t.status_pesanan = 'Selesai'
AND t.status_pembayaran = 'Dibayar' AND p.status = 'aktif';
```

Atau:

```
cd d:\PROJECT\rekomendasi_produk_laravel\python python -c "from cf.data_loader import
load_transaction_data; load_transaction_data()"
```

Akan mencetak misalnya: [OK] Data dimuat: N baris (U user, P produk).

6. Alur perhitungan (dari data sampai metrik)

Transaksi Selesai+Dibayar + produk aktif ■ ▼ Interaksi biner per user-produk (pernah beli = 1) ■ ▼
Leave-last-item holdout per user • urutkan: tanggal → id_transaction → id_product • item holdout =
produk terakhir (1 produk uji) • sisanya = data latih ■ ▼ Bangun cosine similarity HANYA dari data
latih ■ ▼ Top-K rekomendasi per user (agregasi prediction score) ■ ▼ Hit? (apakah item holdout ada
di Top-K) ■ ▼ Precision / Recall / F1 / Hit Rate / Catalog Coverage

7. Rumus-rumus inti

7.1 Interaksi biner

```
x_{ui} = \begin{cases} 1 & \text{jika pelanggan } u \text{ pernah membeli produk } i \\ & \text{pada transaksi valid} \\ 0 & \text{selain itu} \end{cases}
```

Pembelian ulang / qty **tidak** menambah bobot.

7.2 Cosine Similarity (biner)

$$\text{sim}(i, j) = \frac{n(i, j)}{\sqrt{n(i) \times n(j)}}$$

Simbol	Arti
$n(i)$	Jumlah pelanggan unik yang membeli produk i

$n(j)$	Jumlah pelanggan unik yang membeli produk j
$n(i, j)$	Jumlah pelanggan yang membeli keduanya (co-occurrence)

Aturan penyimpanan:

- Pasangan disimpan hanya jika $n(i, j) \geq \text{CF_MIN_CO_OCCURRENCE}$ (default **2**)
- Diagonal $\text{sim}(i, i)$ tidak dipakai
- Matriks simetris: $\text{sim}(i, j) = \text{sim}(j, i)$
- **Tidak** dinormalisasi terhadap skor maksimum katalog

Contoh manual: $n(i)=4, n(j)=2, n(i, j)=2$

$$\text{sim}(i, j) = \frac{2\sqrt{4 \times 2}}{\sqrt{4} + \sqrt{2}} = \frac{2\sqrt{8}}{\sqrt{4} + \sqrt{2}} = 0{,}707107$$

7.3 Prediction score (skor rekomendasi)

Untuk kandidat j dan riwayat latih pelanggan H_u :

$$\text{score}(u, j) = \frac{1}{|H_u|} \sum_{i \in H_u} \text{sim}(i, j)$$

Syarat kandidat:

- belum pernah dibeli pelanggan u
- skor > 0

Urutan peringkat (tie-break):

1. score tertinggi
2. co-occurrence / sisi pendukung tertinggi
3. `id_product` menaik

7.4 Leave-last-item holdout

Untuk setiap user dengan ≥ 2 produk berbeda:

- **Test:** 1 produk terakhir (kemunculan pertama paling akhir menurut waktu)
- **Train:** semua produk sebelum itu
- Similarity dibangun **hanya** dari train → anti data leakage

Karena test = **satu** item, untuk setiap user:

$$\text{hit}_u = \begin{cases} 1 & \text{jika item holdout ada di Top-K} \\ 0 & \text{jika tidak} \end{cases}$$

7.5 Metrik evaluasi (yang diisi ke Tabel 8)

Untuk setiap user u , lalu dirata-ratakan (macro average):

Precision@K

$$P@K(u) = \frac{\text{hit}_u\{K\}}{K} \quad ; \quad P@K = \frac{1}{|U|} \sum_{u \in U} P@K(u)$$

Recall@K (karena $|\text{test}| = 1$)

$$R@K(u) = \frac{\text{hit}_u\{1 = \text{hit}_u\}}{K} \quad ; \quad R@K = \frac{1}{|U|} \sum_{u \in U} R@K(u)$$

Hit Rate@K

$$\text{HitRate@K} = \frac{1}{|U|} \sum_{u \in U} \text{hit}_u$$

Pada skema leave-last-item: **Recall@K = Hit Rate@K.**

F1@K

$$F1@K = \frac{2 \times P@K \times R@K}{P@K + R@K} \quad \text{atau} \quad \begin{cases} 0 & \text{jika } P+R=0 \end{cases}$$

Catalog Coverage@K

$$\text{CatalogCoverage@K} = \frac{|\{\text{produk unik yang muncul di Top-K}\}|}{|\{\text{katalog}\}|} \times 100\%$$

$|U|$ = jumlah users dievaluasi (kolom **Users** di Tabel 8).

7.6 Contoh hitungan kecil (satu user)

Misal $K=5$, user punya holdout produk **30**, Top-5 = {10, 20, 30, 40, 50}.

- $\text{hit}_u = 1$
- $P@5 = 1/5 = 0{,}2$
- $R@5 = 1$
- $F1@5 = 2 \times 0{,}2 \times 1 / (0{,}2 + 1) = 0{,}333\dots$

Jika 100 user dan 10 di antaranya hit:

- $\text{Hit Rate@5} = 10/100 = 0{,}10$ (= 10%)

- $\text{Precision@5} = \frac{0\{, \}10}{5} = \frac{0\{, \}02}{5} (= 2\%)$
- $\text{Recall@5} = \frac{0\{, \}10}{10} (= 10\%)$

8. Interpretasi singkat (untuk pembahasan)

Temuan	Arti
Precision kecil	Normal jika katalog besar; paling banyak 1/K per user saat hit
Recall = Hit Rate	Konsekuensi 1 item holdout
Catalog Coverage tinggi	Rekomendasi menyebar ke banyak produk (diversifikasi), bukan bukti akurasi
Users kecil	Data valid kurang / sedikit user punya ≥ 2 produk

9. Yang tidak diisi dari evaluator ini

Modul `python/cf/evaluator.py` **tidak** menghitung:

- NDCG@K
- Most Popular baseline
- Paired bootstrap / interval kepercayaan
- Acuan peluang acak

Jangan mengisi kolom tersebut dari output evaluator produksi.

10. Checklist cepat

- ☐ DB punya transaksi Selesai + Dibayar
 - ☐ `pip install -r python/requirements.txt`
 - ☐ Env `DB_*` sudah di-set
 - ☐ `python -m cf.evaluator` sukses (ada baris `[EVAL] ...`)
 - ☐ Tabel 8 terisi untuk K=5 dan K=10
 - ☐ Tabel 9 terisi (tanggal, commit, jumlah data, users, durasi)
 - ☐ Format desimal/persen konsisten
 - ☐ Cuplikan terminal / query `evaluation_logs` dilampirkan di lampiran skripsi
-

11. Referensi kode

Komponen	File
Filter transaksi valid	python/cf/c
Holdout + metrik	python/cf/e
Cosine similarity	python/cf/s
Serving rekomendasi PHP	app/Service
Ambang co-occurrence	CF_MIN_CO_O (default 2) di python/conf
)	